Spring MVC Security - JDBC Authentication - Plain Text – Overview

In this lesson, we'll delve into using Spring Security with user accounts stored in a database. Previously, we used hardcoded user accounts in our Java source code as an initial approach to keep things straightforward. However, it's now time to integrate database access.

Integrating with a database is a robust feature of Spring Security, and we'll explore it throughout this this lesson. Let's quickly refresh our memory about user roles. We've used three users in previous lessons: John, Mary, and Susan, complete with passwords and roles. Our task now is to migrate this data into a database.

User ID	Password	Roles
john	test123	EMPLOYEE
mary	test123	EMPLOYEE, MANAGER
susan	test123	EMPLOYEE, MANAGER, ADMIN

Spring Security provides an out-of-the-box solution to fetch user information from a database. By default, it expects you to adhere to its predefined table schemas. The advantage here is that if you follow these schemas, Spring Security seamlessly integrates the required JDBC code, minimizing the Java code you need to author for database interactions. All that's needed from you is to correctly configure it and create the appropriate tables; Spring Security takes care of the rest.

However, you're not restricted to this. If your project requires custom tables, you can design your own schemas. In such cases, you'll need to develop the code to fetch the data, whether through raw JDBC or tools like Hibernate. This means reading both account information and user roles.



In our journey, we'll begin with Spring Security's predefined schemas. It simplifies our integration with the database, handling most of the intricacies on our behalf.

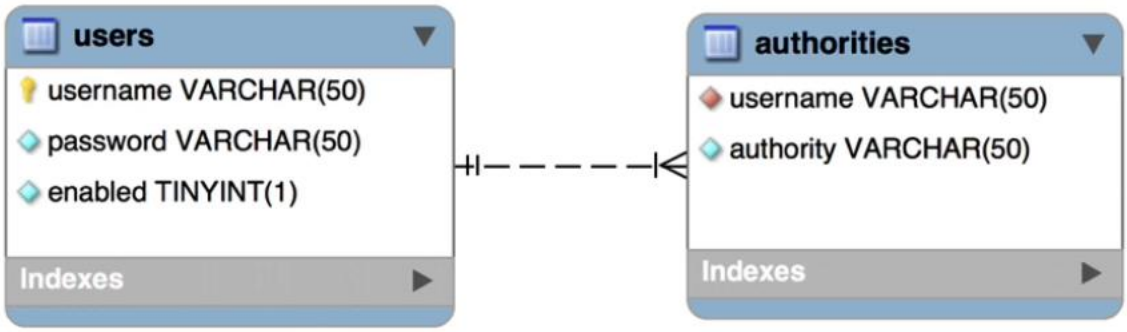
The Development Process Overview:

1. **SQL Script Creation**: We start by drafting an SQL script to establish our database tables.
2. **Database Support Integration**: We'll then add this support in our Maven POM file.
3. **JDBC Properties File**: Next, we'll construct the JDBC properties file.
4. **Spring Security Configuration**: Finally, we'll revise our Spring Security configuration, enabling it to use JDBC for both authentication and authorization tasks.

Spring Security's default schema demands two tables: `'users'` and `'authorities'`. They should have the precise names and specific columns:

- **`'users'`** needs columns: `'username'`, `'password'`, and `'enabled'`.
- **`'authorities'`** requires columns: `'username'` and `'authority'`.

It's vital to adhere to these naming conventions. When discussing `'authorities'`, it essentially refers to what we commonly term `'roles'`.



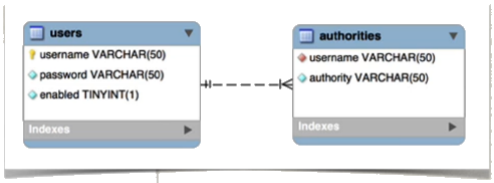
Creating the SQL Script:

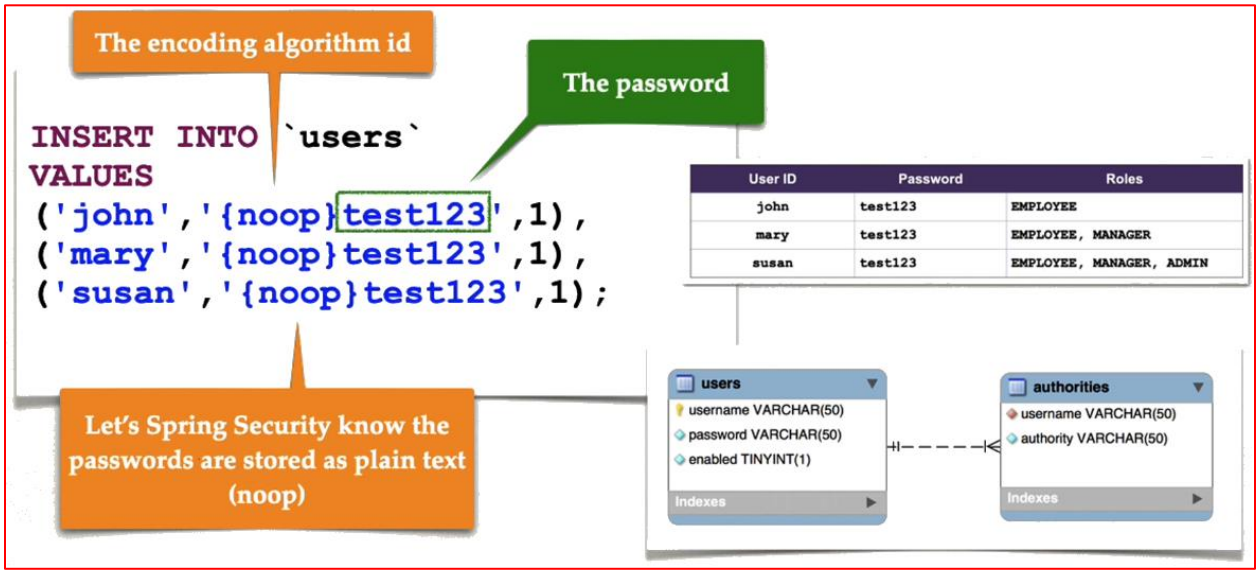
Our first task is to set up our tables: `'users'` and `'authorities'`. The `'users'` table has columns: `'username'`, `'password'`, and `'enabled'`. A primary key will be based on `'username'`. For password storage, we'll begin with plaintext (for simplicity) and then shift towards `bcrypt` encryption in subsequent lessons.

```
CREATE TABLE `users` (
  `username` varchar(50) NOT NULL,
  `password` varchar(50) NOT NULL,
  `enabled` tinyint NOT NULL,

  PRIMARY KEY (`username`)

) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```



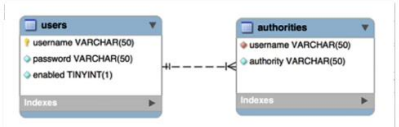


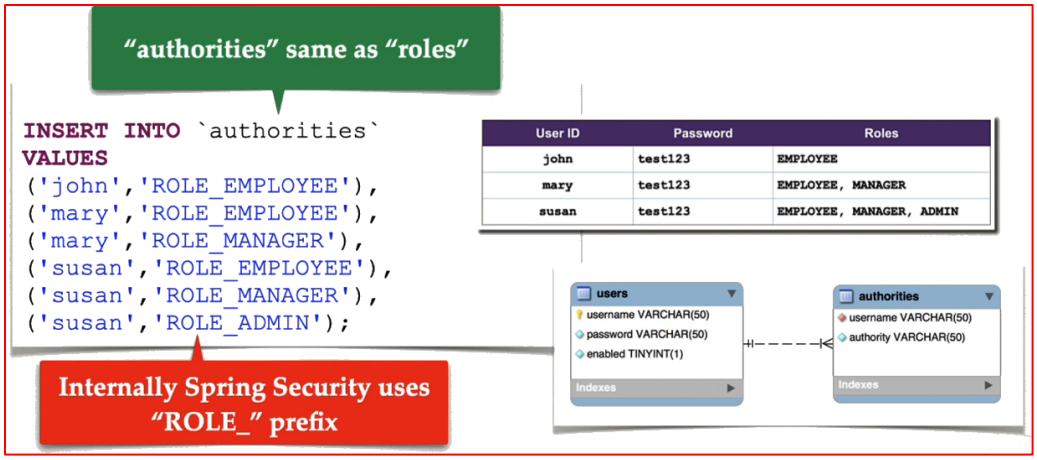
The `'authorities'` table holds user roles. It requires `'username'` and `'authority'` columns, with a unique key set on these. A foreign key constraint ensures the `'username'` in `'authorities'` matches one in the `'users'` table.

```
CREATE TABLE `authorities` (
  `username` varchar(50) NOT NULL,
  `authority` varchar(50) NOT NULL,

  UNIQUE KEY `authorities_idx_1` (`username`,`authority`),

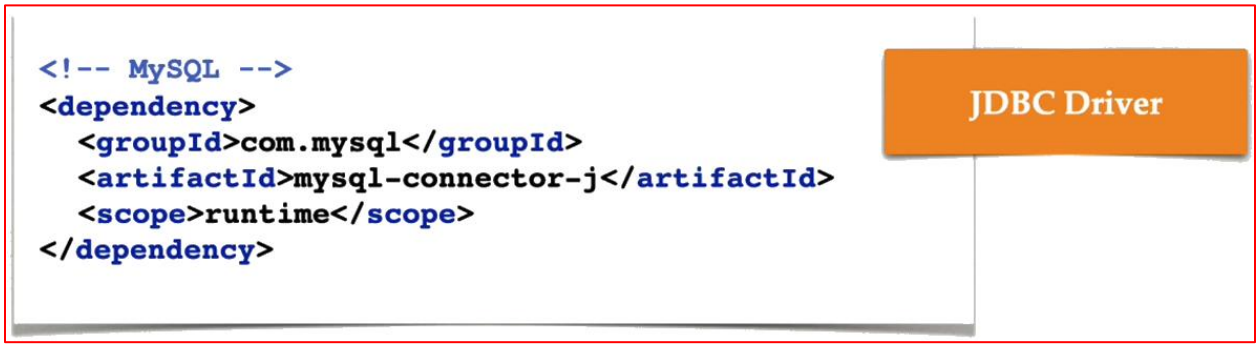
  CONSTRAINT `authorities_ibfk_1`
  FOREIGN KEY (`username`)
  REFERENCES `users` (`username`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```





Maven POM File Configuration:

For database support, we'll reference the JDBC driver in the Maven POM file. We'll use MySQL for our demonstrations.



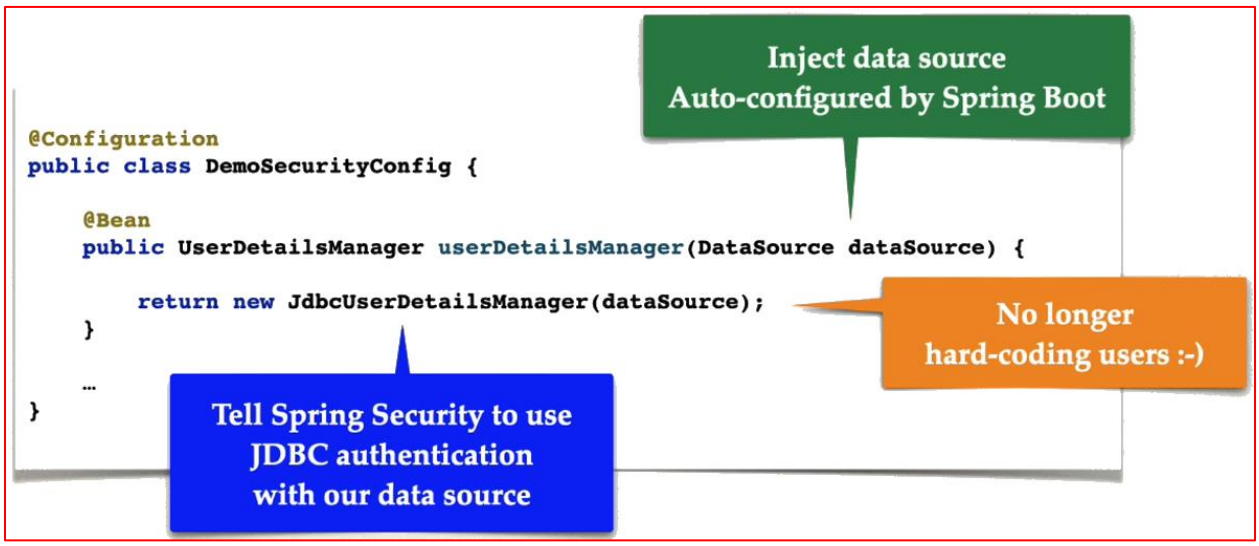
JDBC Properties File Configuration:

Given our singular database schema for the employee directory, we'll store security tables within this same schema, allowing us to reuse our existing properties.



In our `'DemoSecurityConfig'`, we instruct Spring Security to utilize JDBC authentication. Here, we inject an auto-configured data source from Spring Boot. Spring Security then employs JDBC authentication with our data source. No longer will we hardcode users, as we'll now fetch users

and roles directly from our database. By adhering to Spring Security's table and column conventions, we leverage its ability to manage the intricate processes behind the scenes.



That was quite comprehensive, wasn't it? But it's crucial groundwork. Let’s move forward! In the upcoming lesson, we'll set up our database, craft the necessary code, and truly harness the power of Spring Security with our database.